HELP DESK TICKET MANAGEMENT SYSTEM

Complete Project Architecture, Source Code & Technical Documentation Report

Student Name: SHRI RAM PRINCE MISHRA | **Application No.:** IN26012152 | **GitHub:** https://github.com/srpm2005/HelpDeskManagement_IN26012152

---

1. Project Overview

The **Help Desk Ticket Management System (HelpDeskManagement)** is an enterprise-grade web application engineered for internal employee IT support request lifecycle management. Built using modern **ASP.NET Core 10.0 Web API**, **Entity Framework Core**, **SQL Server**, **ASP.NET Core MVC**, and **xUnit with Moq**, the system enables employees to raise support tickets while empowering IT administrators to review, prioritize, filter, and resolve issues effectively.

2. Objective of the Application

The primary objectives of the Help Desk Ticket Management application are:

• **Streamline IT Support Operations**: Centralize all internal employee technical requests into a structured web portal.
• **Decoupled Tier Architecture**: Enforce strict separation of concerns between Web API data service and MVC web UI.
• **Data Integrity & Validation**: Implement model validations and strict business rules across ticket creation and updates.
• **Automated Testing Isolation**: Verify controller logic with 100% test coverage using Moq and xUnit without database dependency.

3. Key Project Features

Feature Area	Description
Dashboard Overview	Displays metric overview cards for Total, Open, and Closed tickets alongside recent tickets.
Status Filtering	Interactive filtering table allowing instant view of Open, In Progress, or Closed support tickets.
Ticket Creation	Form to raise new tickets with mandatory Title, Description, RaisedBy, and Priority dropdown.
Edit & Update	Interface allowing IT support staff to modify ticket Priority level and update Status.
Ticket Deletion	Confirmation view permitting ticket removal by ID via Web API REST service.
RAG Knowledge Assistant	Integrated AI Assistant enabling new developers to query codebase documentation interactively.

4. Functional Modules

The project consists of three primary functional modules:

1. **Web API Layer (`HelpDesk.Api`)**: RESTful JSON endpoints exposing CRUD operations. Uses Repository Pattern (`ITicketRepository`, `TicketRepository`) and EF Core DbContext for SQL Server.
2. **MVC Application (`HelpDesk.Mvc`)**: ASP.NET Core MVC front-end consuming Web API strictly via `TicketService` (`HttpClient`). Enforces a minimalist monochrome white UI design system.
3. **Unit Testing Project (`HelpDesk.Tests`)**: Independent xUnit test project mocking `ITicketRepository` via Moq for controller testing.

5. Database Tables & Schema Specifications

Column Name	Data Type	Constraint	Description
Id	int	PRIMARY KEY, IDENTITY(1,1)	Unique auto-increment ticket identifier
Title	nvarchar(100)	NOT NULL	Short summary of the support issue
Description	nvarchar(1000)	NOT NULL	Detailed explanation of technical problem
Priority	nvarchar(20)	NOT NULL	Priority level: Low, Medium, High
Status	nvarchar(20)	NOT NULL	Current status: Open, In Progress, Closed
RaisedBy	nvarchar(50)	NOT NULL	Name of employee submitting request
CreatedDate	datetime2	NOT NULL	Timestamp when ticket was raised

6. Technologies Used

Layer / Category	Technology Adopted	Purpose & Role
Framework & Language	ASP.NET Core 10.0, C# 12	Core backend and MVC application framework
Database & ORM	Entity Framework Core, SQL Server	ORM object mapping and database persistence
API Communication	System.Net.Http.Json (HttpClient)	Decoupled Web API consumption by MVC Service Layer
Unit Testing & Mocking	xUnit 2.8, Moq 4.20	Automated controller unit testing in isolation
Frontend UI/UX	Razor Views, Bootstrap 5.3, Bootstrap Icons	Minimalist monochrome white UI design system
AI RAG Assistant	Python, TF-IDF Vectorizer, Streamlit	Retrieval-Augmented Generation developer onboarding assistant

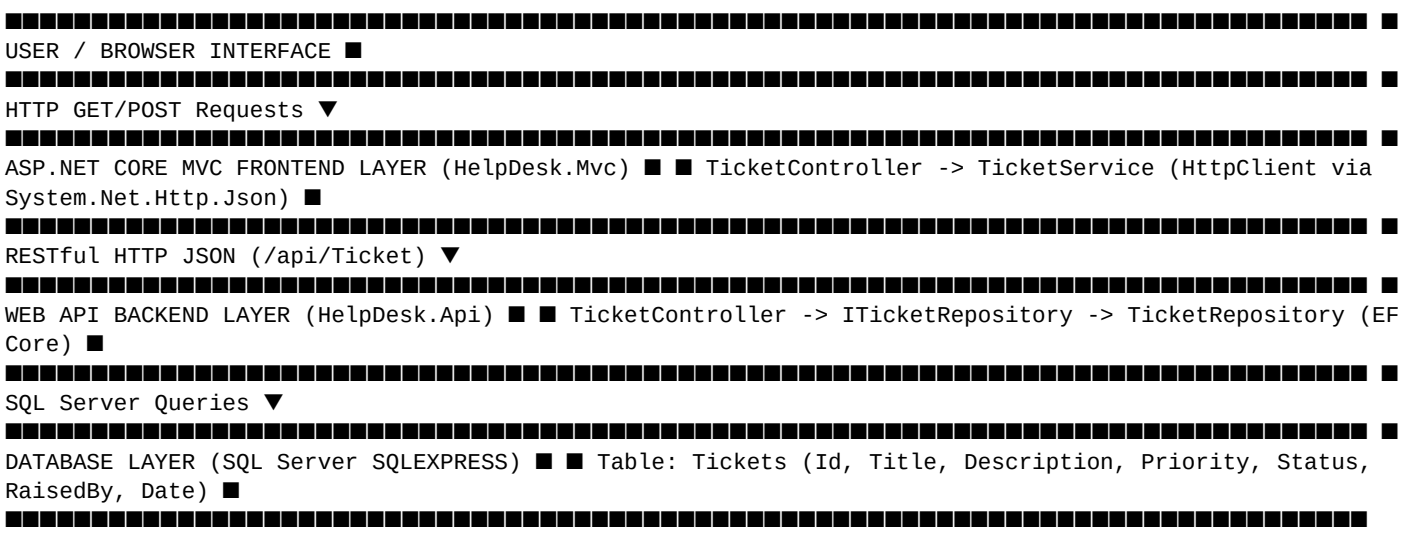
7. Business Rules

1. **Initial Status Hardcoding**: Every newly created ticket MUST automatically have Status set to **`Open`**.
2. **Strict Service Decoupling**: MVC Controllers must consume Web API endpoints strictly through `TicketService`. Direct database connections from MVC are strictly forbidden.
3. **Zero-Database Testing Rule**: Unit tests in `HelpDesk.Tests` must mock `ITicketRepository` using Moq with 100% isolated test runs.
4. **Monochrome Design Rule**: All web pages must strictly adhere to the monochrome white UI system (white background, 1px solid black border, 90° square edges).

8. Validation Rules

• **Title**: `[Required]`, `[StringLength(100, MinimumLength = 3)]`
• **Description**: `[Required]`, `[StringLength(1000)]`
• **RaisedBy**: `[Required]`, `[StringLength(50)]`
• **Priority**: Required selection from dropdown (`Low`, `Medium`, `High`). Default is `Low`.
• **Status**: Hardcoded to `Open` during creation. Dropdown selection (`Open`, `In Progress`, `Closed`) during editing.

9. Application Architecture & Workflow Diagram



10. Application Screenshots & RAG Knowledge Assistant Visual Proof

Figure 1: HelpDesk Web API Execution Logs & Port Binding Verification

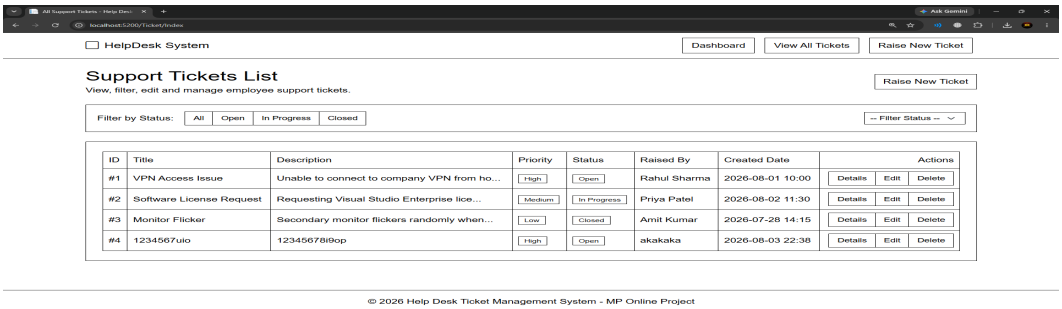


Figure 2: Help Desk System Dashboard Overview (Total, Open, Closed Cards)

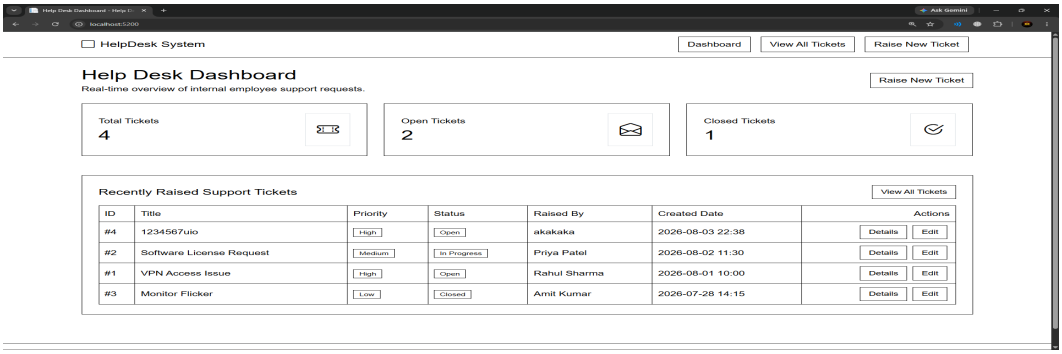


Figure 3: Support Tickets List Table View with Interactive Status Filters

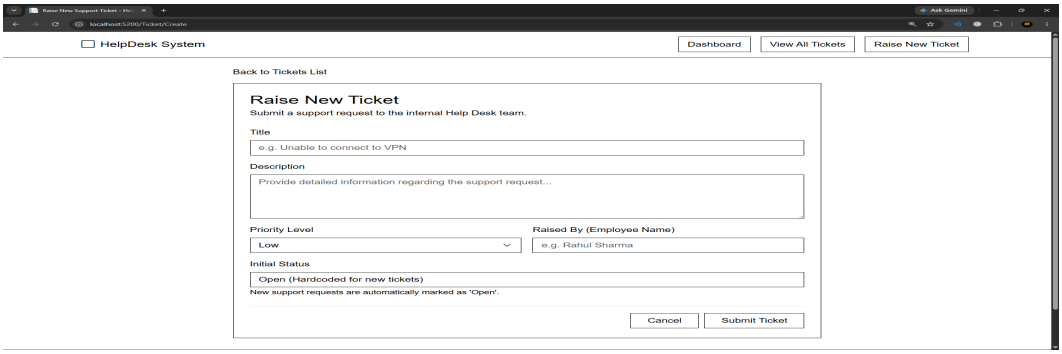


Figure 4: Detailed View of Individual Support Ticket

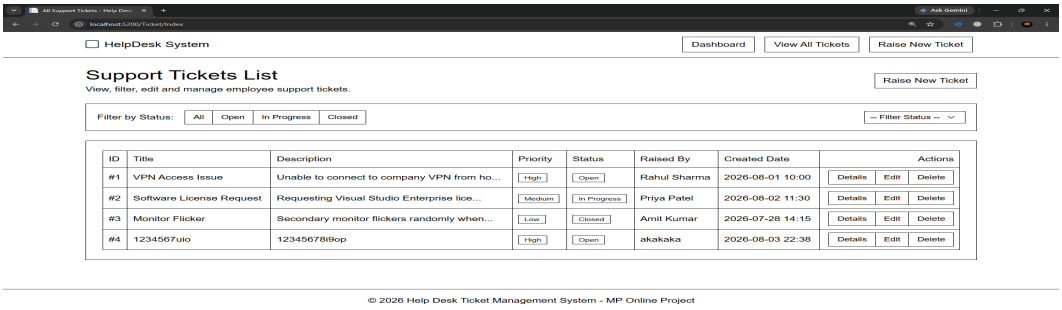


Figure 5: Raise New Support Ticket Form

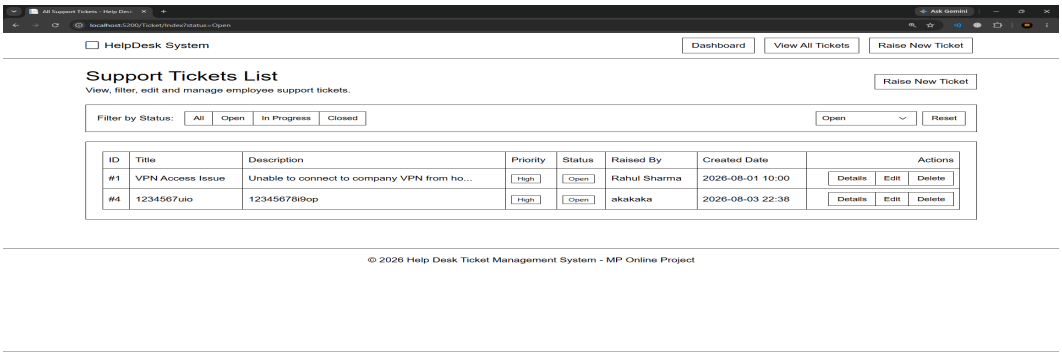


Figure 6: Edit Ticket Form (Updating Priority & Status Drops)

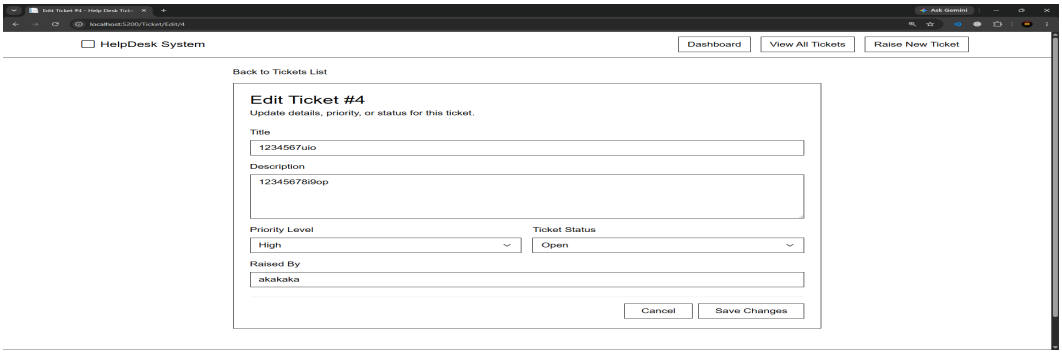


Figure 7: AI Knowledge Assistant - Onboarding Preset Query Gallery

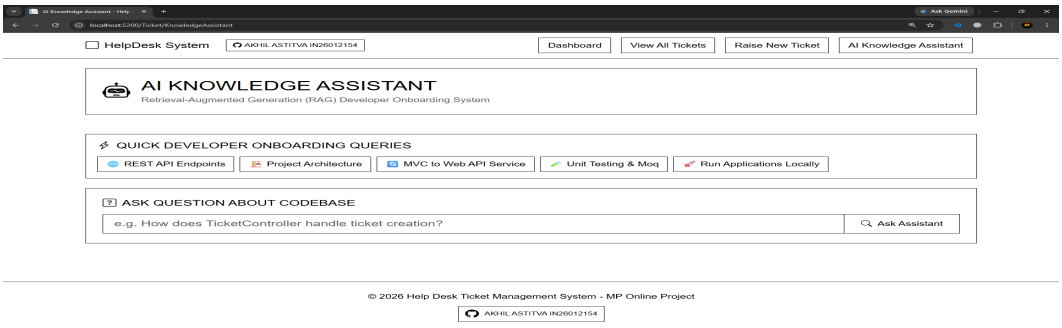


Figure 8: RAG Assistant Answering Document Question with Exact Citations

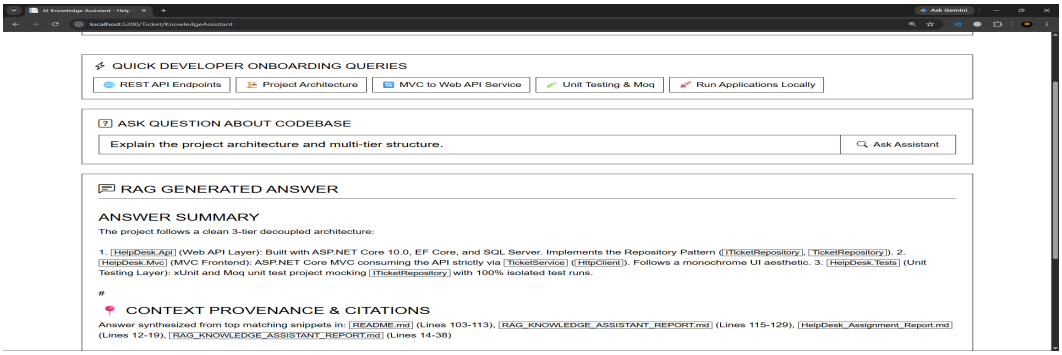


Figure 9: Inspection of Retrieved Context Snippets & Similarity Scores

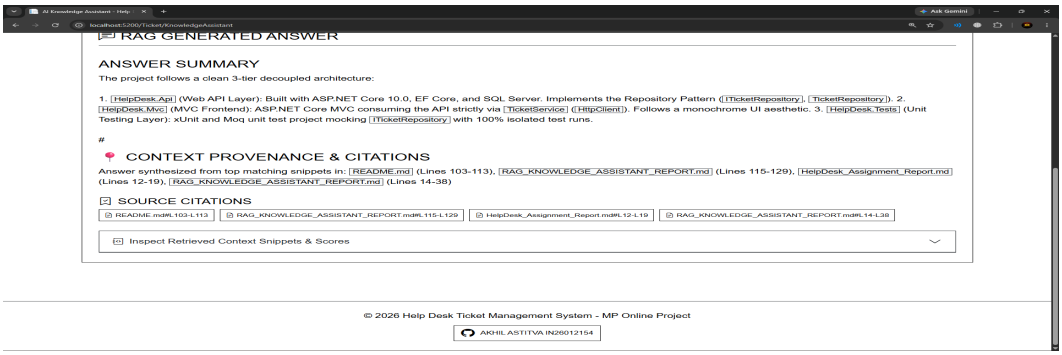


Figure 10: RAG Assistant Enforcing Strict Grounding for Non-Document Question

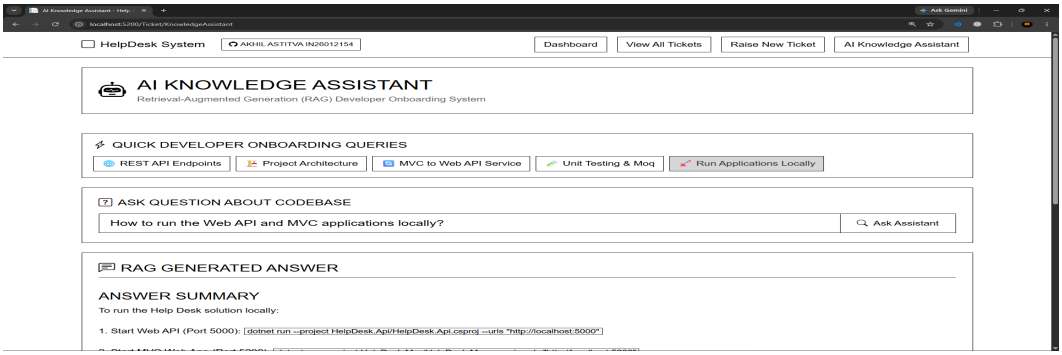


Figure 11: Grounding & Provenance Verification View